VISVESVARAYA TECHNOLOGICAL UNIVERSITY
BELAGAVI-590018

BCS586

MINI PROJECT REPORT
Culturally Sensitive Explainable-AI Based Women's Life-cycle Health Management System

Submitted in partial fulfillment of the requirements for 5th Semester in
Bachelor of Engineering in Computer Science and Engineering
of Visvesvaraya Technological University, Belagavi

Submitted by

AAMINA MEHER A	1RN23CS002
CHAITANYA M	1RN23CS050
B MANASA	1RN23CS039
CHAITHANYA	1RN23CS051

Under the Guidance of :

Ms. Kavitha B

Associate Professor

Dept. of CSE, RNSIT

ESTD : 2001
An Institute with a Difference

Department of Computer Science and Engineering

RNS Institute of Technology

(Accredited by NBA upto 30-06-2025)

Channasandra, Dr. Vishnuvardhan Road, Bengaluru-560098

2025-2026

RNS INSTITUTE OF TECHNOLOGY

Channasandra, Dr. Vishnuvardhan Road, Bengaluru-560098

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

(Accredited by NBA upto 30-06-2028)

ESTD : 2001

An Institute with a Difference

CERTIFICATE

Certified that the Mini Project work has been successfully carried out by **AAMINA MEHER A (1RN23CS002),CHAITANYA M(1RN23CS050),B MANASA(1RN23CS039)** and **CHAITHAYA(1RN23CS051)** bonafide student of **RNS Institute of Technology** in partial fulfillment of the requirements of 5th Semester in **Bachelor of Engineering in Computer Science and Engineering of Visvesvaraya Technological University, Belagavi** during the academic year **2025-2026**. The mini project report has been approved as it satisfies the academic requirements in respect of Mini Project work for the said degree.

Ms.Kavitha B	**Dr. Kavitha C**	**Dr. Ramesh Babu H S**
Associate Professor	Dean and HOD	Principal
Dept. of CSE	Dept. of CSE	RNSIT
RNSIT	RNSIT	

Abstract

Kahaani is an interactive, AI-assisted storytelling and guidance platform designed to empower users—particularly women—by transforming their symptoms, preferences, behaviours, and everyday experiences into meaningful narrative insights. The system blends Natural Language Processing (NLP), lightweight machine-learning models, and structured interaction design to convert complex health-related information into simple, relatable, and culturally grounded stories. Through this narrative medium, Kahaani creates a safe, empathetic space where users can express concerns more comfortably, reflect on their wellbeing, and receive personalised guidance without feeling overwhelmed by medical terminology or diagnostic pressure.

At its core, Kahaani integrates multiple intelligent modules, including text preprocessing pipelines, sentiment and mood recognition, contextual prompt generation, and basic risk-aware recommendation logic. These modules work together to generate dynamic storylines that adapt to the user's emotional tone, historical interactions, and self-reported patterns. The system also maintains structured logs of user inputs—symptoms, reflections, mood markers, and daily events—to support story continuity and help users recognise trends over time. Each narrative produced by Kahaani aims to be gentle, non-judgmental, and supportive, ensuring that users receive encouragement and clarity rather than clinical evaluation.

The conceptual foundation of Kahaani draws from research on women's health challenges such as **endometriosis**, **PCOS and its comorbidities**, **menstrual health literacy**, and **postpartum mental-health interventions**. These insights shape the app's user-facing content, ensuring cultural sensitivity, emotional safety, and evidence-informed relevance. While Kahaani does not provide medical advice or diagnostic claims, its design promotes awareness, early help-seeking, and improved self-understanding by presenting complex topics through stories, metaphors, and intuitive prompts that resonate with users' lived experiences.

Overall, Kahaani serves as a creative and accessible medium for wellbeing engagement. By weaving together narrative interaction, mood sensing, personalised prompts, and lightweight analytics, it enables users to track patterns, explore emotions, and gain gentle guidance in a format that feels human, relatable, and supportive. Through this approach, Kahaani aspires to enhance emotional comfort, promote reflective health behaviour, and build a bridge between personal storytelling and everyday self-care.

Acknowledgement

At the very onset, I would like to place on record my gratitude to all those people who have helped me in making this Mini Project work a reality. Our Institution has played a paramount role in guiding in the right direction.

I would like to profoundly thank **Sri. Satish R Shetty**, Managing Director, RNS Group of Companies, Bengaluru for providing such a healthy environment for the successful completion of this Mini Project work.

I would like to thank our beloved Director, **Dr. M K Venkatesha**, for his constant encouragement which motivated me to complete this work.

I would also like to thank our beloved Principal, **Dr. Ramesh Babu H S** , for providing the necessary facilities to carry out this work.

I am extremely grateful to **Dr. Kavitha C**, Dean and HOD, Department of CSE for her constant encouragement and motivation which helped me to accomplish this work.

I would like to express my sincere thanks to our Mini Project Coordinator, **Sushmitha S**, Assistant Professor , Department of CSE, RNSIT.

My heartfelt thanks to my guide **Ms. Kavitha B** Associate Professor, Department of CSE, for his continuous guidance and constructive suggestions for this work.

I am thankful to all the teaching and non-teaching staff members of the Computer Science and Engineering Department for their encouragement and support throughout this work.

AAMINA MEHER A	**1RN23CS002**
CHAITANYA M	**1RN23CS050**
B MANASA	**1RN23CS039**
CHAITHANYA	**1RN24CS051**

EVALUATION SHEET

SL. NO	PARTICULARS	MAX MARKS	MARKS AWARDED
1	Presentation	10	
2	Project Report	50	
3	Implementation	15	
4	Question and Answer	25	
	TOTAL	100	

Signature of Guide

Contents

Chapter 1

Introduction

1.1 Background:

The background of **Kahaani** emerges from the increasing demand for accessible, intuitive, and culturally attuned digital tools that assist individuals—particularly women—in understanding and articulating their health experiences. Across many communities, users often face barriers such as limited health literacy, language constraints, stigma around discussing sensitive symptoms, and difficulty recognising early signs of conditions like PCOS, endometriosis, and postpartum mental-health disturbances. Despite the availability of digital health platforms, many of them rely on clinical terminology, rigid interfaces, or diagnostic-oriented workflows that feel intimidating and inaccessible to everyday users. As a result, individuals who most need personalised guidance and emotional support often remain underserved by existing technological solutions.

Recent advancements in **Natural Language Processing (NLP)**, sentiment analysis, and AI-based conversational interfaces have opened new possibilities for translating complex healthcare concepts into simplified, engaging, and human-centric interactions. These developments inspired the conceptual foundation of Kahaani—a system that leverages storytelling as a powerful medium for self-expression, reflection, and early awareness. By converting user-provided inputs such as symptoms, moods, or daily experiences into personalised narratives, Kahaani reduces cognitive load, enhances comfort, and encourages users to articulate concerns they may struggle to express in conventional clinical settings. The inclusion of lightweight recommendation logic and emotion-aware interaction design further enables the system to provide context-sensitive prompts, reassurance, and gentle suggestions aligned with users' evolving needs.

Thus, the background of Kahaani lies in addressing persistent gaps within the digital wellbeing ecosystem by fusing technology, narrative psychology, and user-centric design principles. The platform builds on insights from mHealth research, women's health studies, behavioural science, and AI-driven support systems to create an experience that is both informative and emotionally resonant. By prioritising empathy, cultural sensitivity, and simplification, Kahaani aims to bridge the communication divide between clinical knowledge and everyday lived experiences. Ultimately, it seeks to offer users a safe, relatable, and empowering way to engage with their wellbeing and make more informed decisions about seeking timely support.

1.2 Problem Statement:

Many individuals, particularly women, face significant challenges in expressing their health concerns clearly, understanding symptom patterns, and accessing supportive guidance. Conditions such as PCOS, endometriosis, and postpartum mental-health issues often go unnoticed or misunderstood due to communication barriers, limited awareness, and the absence of relatable,

user-friendly tools. Existing digital platforms either provide generic information or rely on complex medical terminology, making them difficult for users to navigate and emotionally engage with. There is a need for an application that simplifies health expression, supports emotional wellbeing, and presents information in a form users can easily understand. Kahaani aims to address this gap by transforming user inputs into meaningful narratives, offering personalised insights, and providing supportive interactions using simple AI and NLP techniques. By using storytelling as a medium, the system strives to make self-expression easier, improve user understanding, and encourage timely helpseeking behaviour.

1.3 Objectives:

- **To develop a user-friendly storytelling system** that converts user inputs into meaningful, personalised narratives.

- **To apply basic NLP techniques** for understanding user prompts, sentiments, and preferences.

- **To provide contextual insights and gentle recommendations** based on user mood, symptoms, and interaction patterns.

- **To support emotional wellbeing** by offering relatable, culturally sensitive content inspired by research on women's health and mental-health interventions.

- **To enable simple data logging and tracking** for better understanding of user behaviour and narrative trends.

- **To create a supportive digital environment** that encourages self-expression and early help-seeking behaviour without replacing clinical diagnosis.

1.4 Scope:

- **Narrative-Based Interaction:**

The system converts user inputs—symptoms, moods, experiences—into simple, personalised stories to enhance comfort and self-expression.

- **Text and Sentiment Processing:**

Includes modules for text preprocessing, sentiment detection, and mood interpretation to contextualise user interactions.

- **Symptom Logging and Pattern Tracking:**

Allows users to record daily symptoms, emotions, and reflections for longitudinal understanding and story continuity.

- **Basic Risk-Aware Recommendations:**

Provides general, non-clinical suggestions and supportive prompts based on identified patterns without making medical claims.

- **User-Centric and Culturally Sensitive Design:**

Ensures language, tone, and story formats are accessible and relatable for diverse user groups, especially women.

- **Adaptive Story Generation:**

Generates narratives that evolve with the user's history, preferences, and interaction patterns.

- **Educational Insights:**

Offers simple explanations about wellbeing topics such as menstrual health, mood patterns, or lifestyle factors, grounded in evidence-informed content.

- **Privacy and Data Security:**

Ensures safe handling of user inputs through structured logging and secure data management practices.

- **Modular Architecture:**

Supports incremental expansion—such as adding new story themes, additional health modules, or enhanced AI models.

- **Non-Clinical Digital Support:**

Functions solely as a wellbeing and guidance companion, not as a diagnostic or treatment tool.

Chapter 2

Methodology

2.1 Software Requirements:

1. Operating System: Windows / Linux / macOS

Kahaani is designed to be platform-independent, enabling development and deployment across major operating systems. Whether the system runs on Windows, Linux distributions, or macOS, all major components—including Python, Flask, and database services—are fully compatible, ensuring flexibility for developers and ease of use for testers.

2. Programming Language: Python

Python serves as the primary programming language due to its simplicity, readability, and extensive ecosystem of libraries supporting Natural Language Processing (NLP), machine learning, data manipulation, and backend development. Its robust community support and seamless integration with Flask make it ideal for building scalable, lightweight applications like Kahaani.

3. Framework: Flask (Backend Web Application)

Flask is used as the backend framework because of its minimalistic, modular structure, which allows developers to build flexible routes, APIs, and application logic without unnecessary overhead. Its lightweight nature is suitable for Kahaani's requirements, enabling quick deployment, easy debugging, and the integration of NLP pipelines and data storage components.

4. Front-End Tools: HTML, CSS, JavaScript

The user interface is developed using standard web technologies:

(a)**HTML** for structuring pages and story components

(b)**CSS** for styling, layout responsiveness, and theme consistency

(c)**JavaScript** for interactive elements, dynamic story rendering, and seamless communication with backend APIs

|

These tools ensure that Kahaani remains browser-accessible and user-friendly, with the potential for future upgrades to modern frameworks like React or Vue.

5. Libraries: NLP and Text-Processing Packages (NLTK, TextBlob, etc.)

Kahaani relies on several libraries to interpret and transform user inputs:

(a)NLTK for tokenization, stop-word removal, and linguistic preprocessing

(b)TextBlob for sentiment analysis and simplified NLP

(c)JSON libraries for structured data handling and API responses
These packages enable mood detection, narrative shaping, and the generation of personalised storylines based on user behaviour and language cues.

6. Database: SQLite

SQLite serves as the lightweight database engine used to store user data, including symptoms, story outputs, interaction history, and feedback. Its file-based architecture makes it easy to set up, portable, and suitable for applications with moderate data requirements. SQLite integrates seamlessly with Flask and Python, simplifying CRUD operations.

7. IDE / Code Editors: Visual Studio Code or PyCharm

For development, editors like **VS Code** and **PyCharm** offer features such as syntax highlighting, intelligent code suggestions, virtual environment management, Git integration, and debugging tools. These environments significantly enhance developer productivity during the implementation of NLP modules, routes, and UI components.

8. Browser Support: Chrome / Edge / Firefox

Kahaani is optimized for compatibility with major modern browsers, including Google Chrome, Microsoft Edge, and Mozilla Firefox. Ensuring cross-browser support guarantees accessibility for users across devices and platforms.

9. Version Control: Git / GitHub

Git is used for version control to manage code changes, collaboration, branching, and issue tracking. GitHub provides a remote repository for collaboration, documentation, and continuous integration if needed. This ensures systematic development, safe backups, and transparent project progress.

2.2 Algorithms:

1. Text Processing Algorithm

This algorithm is responsible for cleaning, structuring, and preparing raw user input before it enters the storytelling pipeline. It uses techniques such as tokenization, stop-word removal, lemmatization, and noise filtering to convert unstructured text into meaningful linguistic units. By removing irrelevant characters, correcting spelling variations, and standardizing phrases, this algorithm ensures that the input is accurate and ready for further analysis. The processed text improves the quality of narratives and enhances the performance of downstream algorithms like sentiment detection and classification.

2. Template-Based Story Generation Algorithm

Kahaani uses a structured, template-driven approach to generate narratives that are coherent and context-aware. This algorithm identifies the appropriate story template by matching user mood, symptoms, or preferences with predefined narrative structures. Once the template is selected, the algorithm dynamically inserts user-specific elements—such as emotions, concerns, or events—into the story framework. This ensures consistency, personalization, and ease of expansion, as new templates can be added without altering the core logic.

3. Classification Algorithm

The classification algorithm identifies the **genre**, **mood**, or **story theme** based on the user's input. Using lightweight machine-learning models or keyword-based rule systems, it categorizes text into groups such as "comforting," "motivational," "exploratory," or "reflective." This classification helps guide the storytelling engine toward the most relevant tone and ensures that the output aligns with the user's psychological and contextual needs. It also assists in organizing user interactions into meaningful categories for pattern recognition.

4. Sentiment Analysis Algorithm

This algorithm plays a crucial role in emotional adaptation. It evaluates the user's text to determine emotional tones such as happiness, stress, frustration, confusion, or sadness. By assigning polarity scores (positive/negative/neutral) and subjectivity levels, the algorithm allows Kahaani to adjust narrative tone accordingly—offering supportive, warm, or encouraging messages that align with the user's emotional state. This helps create more empathetic and human-like interaction.

5. Recommendation Algorithm

Kahaani includes a recommendation system that suggests suitable story genres, themes, styles, or reflective prompts based on user history, inferred preferences, or popular interaction patterns. The algorithm analyzes past inputs, story engagement patterns, and feedback ratings to identify what the user enjoys or responds well to. This ensures that future interactions feel increasingly personalized and engaging. The recommendations remain non-clinical and strictly oriented toward storytelling and wellbeing support.

6. Feedback Handling Algorithm

This algorithm manages user feedback to continuously improve the system. When users rate stories or interaction experiences, the algorithm records these responses in the database and updates preference models or recommendation patterns. It helps refine future narrative generation by recognizing which styles or tones resonate most effectively. Over time, this creates a loop of continuous learning and personalization, enhancing the overall user experience while maintaining simplicity and safety.

2.3 Data Collection and Analysis procedure
Data Collection Procedure (Elaborated)

(a)User Input Acquisition
The system collects various forms of user input—such as text prompts, symptom descriptions, emotional reflections, preferred story genres, and specific story requests—through an intuitive application interface. These inputs form the foundation of the storytelling process.

(b)Structured Data Storage
Once collected, all inputs are securely stored in a database (e.g., SQLite). Data is saved in a structured format with fields such as timestamp, input text, user mood, selected category, and interaction metadata. This organization ensures easy retrieval and efficient processing during narrative generation.

(c)Feedback Logging
Users may optionally provide ratings, comments, or reactions to the stories generated by Kahaani. This feedback is also recorded in the database as part of the user experience dataset. It serves as an important signal for refining templates, improving personalization, and enhancing future recommendations.

(d)Anonymization and Privacy Protection
To maintain ethical standards and protect user identity, all collected information is anonymized. User identifiers are removed or replaced with system-generated tokens, ensuring that stored data cannot be traced back to individual users. This supports safe analysis and responsible data handling.

(e)Data Organization and Maintenance
The dataset is periodically organized and validated to ensure consistency. Duplicate entries, incomplete fields, or erroneous values are resolved to maintain data integrity. This careful maintenance ensures reliable results during later analysis and system evaluation.

Data Analysis Procedure (Elaborated)

(a)Preprocessing and Text Cleaning
Collected user inputs undergo a preprocessing pipeline involving tokenization, stop-word removal, noise filtering, and normalization. This step removes unnecessary characters and standardizes the text, enabling more accurate sentiment detection and story generation.

(b)Feature Extraction and Pattern Identification
Key linguistic and emotional features—such as sentiment polarity, mood indicators, dominant keywords, and genre cues—are extracted from the cleaned text. These features help classify the input, detect user intentions, and guide the selection of suitable story templates and tones.

(c)Performance and Behavioural Analysis
Story generation outcomes, along with user feedback (e.g., ratings, comments, or engagement history), are analyzed to identify system strengths and areas for improvement. This includes detecting which story formats are most popular, which tones resonate best, and where users might feel less satisfied.

(d)Trend and Preference Evaluation
By examining recurring patterns across inputs, the system identifies common user preferences, frequently requested themes, and emotional trends. These insights help tailor future interactions and ensure the system evolves with user needs.

(e)System Refinement and Optimization
Based on analytical findings, narrative templates are updated, sentiment rules are fine-tuned, and the recommendation logic is improved. Enhanced responsiveness and personalization contribute to a more intuitive and emotionally supportive user experience.

Chapter 3

Implementation

3.1 Code:

PAGES

(a)AI COMPANION

```
import React, { useState, useEffect, useRef } from "react";
import { base44 } from "@/api/base44Client";
import { useQuery } from "@tanstack/react-query";
import { Card } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Textarea } from "@/components/ui/textarea";
import { Badge } from "@/components/ui/badge";
import { Send, Loader2, Heart } from "lucide-react";
import ReactMarkdown from "react-markdown";
import { differenceInDays } from "date-fns";

export default function AICompanion() {
  const [messages, setMessages] = useState([]);
  const [input, setInput] = useState("");
  const [isLoading, setIsLoading] = useState(false);
  const [phase, setPhase] = useState(null);
  const initialized = useRef(false);
  const bottomRef = useRef();

  // --- Fetch Data ---
  const { data: cycleLogs = [] } = useQuery({
    queryKey: ["cycleLogs"],
    queryFn: () => base44.entities.CycleLog.list("-date", 30),
```

```
});

const { data: mood } = useQuery({
  queryKey: ["recentMood"],
  queryFn: async () => (await base44.entities.MoodEntry.list("-date", 1))[0],
});

// --- Detect Phase ---
useEffect(() => {
  const p = cycleLogs.find(l => l.is_period_day);
  if (!p) return;
  const d = differenceInDays(new Date(), new Date(p.date));
  setPhase(d <= 5 ? "menstrual" : d <= 13 ? "follicular" : d <= 16 ? "ovulation" : "luteal");
}, [cycleLogs]);

// --- Greeting ---
useEffect(() => {
  if (!initialized.current && phase) {
    const g = {
      menstrual: "Hello dear one 🌙 This is a tender time. I'm here with you.",
      follicular: "Namaste 💗 Your energy is rising—what's on your mind?",
      ovulation: "Hello radiant soul ✨ You're glowing today. How do you feel?",
      luteal: "Hi love 💗 Emotions may be heavier now. I'm here to hold space."
    }[phase];
    setMessages([{ role: "assistant", content: g }]);
    initialized.current = true;
  }
}, [phase]);

// --- Auto-scroll ---
```

```
 useEffect(() => bottomRef.current?.scrollIntoView({ behavior: "smooth" }), [messages]);


 // --- LLM Prompt (Ultra-Condensed) ---
 const buildPrompt = (history, userMessage) => `
You are **Kahaani**: a gentle, cycle-aware, trauma-informed companion.
Tone: warm, validating, culturally sensitive, 3–5 sentences max.
Always:
- Reflect emotion first
- Normalize cycle-phase effects without dismissing feelings
- Offer one simple grounding tool if needed
- End with a soft question or affirmation

User Phase: ${phase || "unknown"}
Mood: ${mood?.mood || "unknown"}

Recent Chat:
${history}

User: ${userMessage}
Respond empathetically.`;

 // --- Send Message ---
 const sendMessage = async () => {
   if (!input.trim()) return;
   const text = input.trim();
   setInput("");
   setMessages(m => [...m, { role: "user", content: text }]);
   setIsLoading(true);

   try {
    const history = messages.slice(-8)
```

```
    .map(m => `${m.role === "user" ? "User" : "Kahaani"}: ${m.content}`)
    .join("\n");

  const response = await base44.integrations.Core.InvokeLLM({
    prompt: buildPrompt(history, text),
  });

  setMessages(m => [...m, { role: "assistant", content: response }]);
 } catch {
  setMessages(m => [...m, {
    role: "assistant",
    content: "I'm having trouble connecting, dear one. Try again soon 🩶"
  }]);
 } finally {
  setIsLoading(false);
 }
};

// --- Enter key ---
const handleKey = e => {
 if (e.key === "Enter" && !e.shiftKey) {
  e.preventDefault();
  sendMessage();
 }
};

// --- Phase UI ---
const phaseInfo = {
 menstrual: "Menstrual Phase",
 follicular: "Follicular Phase",
 ovulation: "Ovulation Phase",
```

```
  luteal: "Luteal Phase"
};

return (
  <div className="min-h-screen lg:ml-64">
    <div className="max-w-4xl mx-auto px-4 py-8">

      {/* Header */}
      <div className="flex justify-between items-center mb-6">
        <div>
          <h2 className="text-3xl font-bold text-[#5C3D5E]">AI Companion</h2>
          <p className="text-purple-400 -mt-1">A sacred space for your story</p>
        </div>
        {phase && <Badge>{phaseInfo[phase]}</Badge>}
      </div>

      {/* Chat Card */}
      <Card className="h-[calc(100vh-240px)] flex flex-col shadow-xl border-none">

        {/* Messages */}
        <div className="flex-1 p-6 overflow-y-auto space-y-4">
          {messages.map((m, i) => (
            <div key={i} className={`flex ${m.role === "user" ? "justify-end" : "justify-start"}`}>
              <div className={`max-w-[80%] px-4 py-3 rounded-2xl ${
                m.role === "user"
                ? "bg-gradient-to-r from-purple-500 to-pink-500 text-white"
                : "bg-purple-50 text-gray-800"
              }`}>
                {m.role === "assistant" && (
                  <div className="flex items-center gap-2 mb-2 text-purple-500 text-xs">
                    <Heart className="w-4 h-4" /> Kahaani
```

```
        </div>
      )}
      <ReactMarkdown>{m.content}</ReactMarkdown>
    </div>
  </div>
 ))}

 {isLoading && (
  <div className="flex">
   <div className="px-4 py-3 bg-purple-50 rounded-2xl flex items-center gap-2">
    <Loader2 className="w-4 h-4 animate-spin text-purple-500" /> Listening...
   </div>
  </div>
 )}

 <div ref={bottomRef} />
</div>

{/* Input */}
<div className="border-t p-4">
 <div className="flex gap-3">
  <Textarea
   value={input}
   onChange={e => setInput(e.target.value)}
   onKeyDown={handleKey}
   placeholder="Share what's in your heart..."
   disabled={isLoading}
   className="min-h-[60px]"
  />
  <Button onClick={sendMessage} disabled={!input.trim() || isLoading}
   className="h-[60px] w-[60px] bg-gradient-to-r from-purple-500 to-pink-500 text-white">
```

```
            {isLoading ? <Loader2 className="animate-spin" /> : <Send />}
          </Button>
        </div>

        <p className="text-xs text-gray-400 mt-2 text-center">
          Supportive companion, not a medical substitute.
        </p>
       </div>

      </Card>
    </div>
   </div>
  );
}
```

(b)MENTAL WELLNESS

```
import React, { useState } from "react";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@/components/ui/tabs";
import { Brain, Heart, Baby, ArrowLeft } from "lucide-react";
import { Link } from "react-router-dom";
import { createPageUrl } from "@/utils";

import PHQ9Screener from "@/components/mental-wellness/PHQ9Screener";
import GAD7Screener from "@/components/mental-wellness/GAD7Screener";
import EPDSScreener from "@/components/mental-wellness/EPDSScreener";
import ScreeningHistory from "@/components/mental-wellness/ScreeningHistory";

export default function MentalHealthScreening() {
  const [activeTab, setActiveTab] = useState("overview");

  const TestCard = ({ icon: Icon, title, desc, tab, color }) => (
```

```
  <Card onClick={() => setActiveTab(tab)} className="bg-white hover:shadow-lg border-none cursor-pointer">
    <CardHeader>
      <CardTitle className="text-lg text-[#5C3D5E] flex items-center gap-2">
        <Icon className={`w-5 h-5 ${color}`} /> {title}
      </CardTitle>
    </CardHeader>
    <CardContent>
      <p className="text-sm text-gray-600 mb-3">{desc}</p>
      <Button size="sm" className={`${color.replace("text", "bg").replace("600", "600 hover:bg-700")} text-white`}>
        Start →
      </Button>
    </CardContent>
  </Card>
 );

 return (
  <div className="min-h-screen lg:ml-64">
    <div className="max-w-4xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
      <Link to={createPageUrl("MentalWellness")}>
        <Button variant="ghost" size="sm" className="mb-4 -ml-2">
          <ArrowLeft className="w-4 h-4 mr-2" /> Back
        </Button>
      </Link>

      <h2 className="text-3xl font-bold text-[#5C3D5E] mb-2">Mental Health Screening</h2>
      <p className="text-purple-400 mb-6">Evidence-based assessments for your wellbeing</p>

      <Tabs value={activeTab} onValueChange={setActiveTab}>
        <TabsList className="grid w-full grid-cols-4 mb-6">
          <TabsTrigger value="overview">Overview</TabsTrigger>
```

```
 <TabsTrigger value="phq9">PHQ-9</TabsTrigger>
 <TabsTrigger value="gad7">GAD-7</TabsTrigger>
 <TabsTrigger value="epds">EPDS</TabsTrigger>
</TabsList>

<TabsContent value="overview">
 <Card className="bg-gradient-to-br from-indigo-50 to-purple-50 border-none mb-6">
  <CardContent className="p-6 text-sm text-gray-700">
   <p className="mb-3">
    Quick, validated mental-health screenings to help understand your mood and patterns.
   </p>
   <p>• 2–5 min • Private • Saved securely • Personalized insights</p>
  </CardContent>
 </Card>

 <div className="grid gap-4 mb-6">
  <TestCard
   icon={Brain}
   title="PHQ-9: Depression"
   desc="9-item assessment for depressive symptoms."
   tab="phq9"
   color="text-indigo-600"
  />
  <TestCard
   icon={Heart}
   title="GAD-7: Anxiety"
   desc="7-item screening for anxiety levels."
   tab="gad7"
   color="text-pink-600"
  />
  <TestCard
```

```
            icon={Baby}
            title="EPDS: Postpartum"
            desc="10-item scale for pregnancy & postpartum mood."
            tab="epds"
            color="text-purple-600"
          />
        </div>

        <ScreeningHistory />
      </TabsContent>

      <TabsContent value="phq9"><PHQ9Screener onComplete={() => setActiveTab("overview")}
/></TabsContent>
      <TabsContent value="gad7"><GAD7Screener onComplete={() => setActiveTab("overview")}
/></TabsContent>
      <TabsContent value="epds"><EPDSScreener onComplete={() => setActiveTab("overview")}
/></TabsContent>

    </Tabs>
   </div>
  </div>
 );
}
```

(c)SUPPORT GROUPS

```
import React, { useState } from "react";
import { base44 } from "@/api/base44Client";
import { useQuery } from "@tanstack/react-query";

import {
 Users, Sparkles, Award, MessageCircle,
 TrendingUp, Shield, Calendar
} from "lucide-react";

import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
```

```
import { Button } from "@/components/ui/button";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@/components/ui/tabs";

import GroupDiscovery from "@/components/support/GroupDiscovery";
import MyGroups from "@/components/support/MyGroups";
import DailyCheckInWidget from "@/components/support/DailyCheckInWidget";
import UserBadges from "@/components/support/UserBadges";

export default function SupportGroups() {
  const [activeTab, setActiveTab] = useState("discover");

  // ------------------------------------------------------------
  // API Calls using base44
  // ------------------------------------------------------------
  const { data: myCircles = [] } = useQuery({
    queryKey: ["myCircles"],
    queryFn: () => base44.entities.Circle.list(),
  });

  const { data: badges = [] } = useQuery({
    queryKey: ["userBadges"],
    queryFn: () => base44.entities.UserBadge.list("-earned_date"),
  });

  const { data: todayCheckIn } = useQuery({
    queryKey: ["todayCheckIn"],
    queryFn: async () => {
      const today = new Date().toISOString().split("T")[0];
      const res = await base44.entities.DailyCheckIn.filter({ check_in_date: today });
      return res[0] || null;
    },
```

```
});

// ---------------------------------------------------------
// Small Reusable UI Components
// ---------------------------------------------------------
const StatCard = ({ title, count, Icon, color }) => (
  <Card className={`bg-gradient-to-br ${color} border-none`}>
    <CardContent className="p-5 flex items-center justify-between">
      <div>
        <p className="text-sm text-gray-600 mb-1">{title}</p>
        <p className="text-2xl font-bold">{count}</p>
      </div>
      <Icon className="w-8 h-8 opacity-70" />
    </CardContent>
  </Card>
);

const SessionCard = ({ title, host, time }) => (
  <div className="flex items-center justify-between p-4 bg-white rounded-xl">
    <div>
      <p className="font-medium text-gray-900">{title}</p>
      <p className="text-xs text-gray-500 mt-1">{host}</p>
      <p className="text-xs text-purple-600 mt-1">{time}</p>
    </div>
    <Button size="sm" className="bg-violet-600 hover:bg-violet-700 text-white">
      Join
    </Button>
  </div>
);

// ---------------------------------------------------------
```

```
// Main UI
// ------------------------------------------------------------

return (
  <div className="min-h-screen lg:ml-64">
    <div className="max-w-6xl mx-auto px-4 py-8">

      {/* ---------------------- Header ---------------------- */}
      <div className="flex items-center gap-3 mb-8">
        <div className="w-12 h-12 rounded-2xl bg-rose-100 flex items-center justify-center">
          <Users className="w-6 h-6 text-rose-600" />
        </div>
        <div>
          <h2 className="text-3xl font-bold text-[#5C3D5E]">Support Circles</h2>
          <p className="text-purple-400">Safe spaces for connection and healing</p>
        </div>
      </div>

      {/* ---------------------- Stats ---------------------- */}
      <div className="grid sm:grid-cols-3 gap-4 mb-6">
        <StatCard
          title="My Circles"
          count={myCircles.length}
          Icon={MessageCircle}
          color="from-rose-50 to-pink-50"
        />

        <StatCard
          title="Badges Earned"
          count={badges.length}
          Icon={Award}
```

```
  color="from-amber-50 to-yellow-50"
 />

 <StatCard
  title="Check-in Streak"
  count="7 days"
  Icon={TrendingUp}
  color="from-teal-50 to-emerald-50"
 />
</div>

{/* ---------------------- Daily Check-in ---------------------- */}
{!todayCheckIn && <DailyCheckInWidget />}

{/* ---------------------- Badges Preview ---------------------- */}
{badges.length > 0 && (
 <Card className="mb-6 bg-gradient-to-r from-purple-50 to-pink-50 border-none">
  <CardContent className="p-5">
   <div className="flex items-center justify-between mb-3">
    <div className="flex items-center gap-2">
     <Sparkles className="w-5 h-5 text-purple-600" />
     <h3 className="font-semibold text-[#5C3D5E]">Your Badges</h3>
    </div>

    <Button variant="ghost" size="sm" onClick={() => setActiveTab("badges")}>
     View All →
    </Button>
   </div>

   <div className="flex gap-2 overflow-x-auto pb-2">
    {badges.slice(0, 5).map((b) => (
```

```
        <div
          key={b.id}
          className="flex-shrink-0 px-3 py-2 bg-white rounded-lg border border-purple-200"
        >
          <span className="text-2xl">{b.icon}</span>
        </div>
       ))}
     </div>
   </CardContent>
  </Card>
)}

{/* ---------------------- Guidelines ---------------------- */}
<Card className="mb-6 bg-blue-50 border-blue-200">
  <CardContent className="p-4 flex items-start gap-3 text-xs text-blue-700">
    <Shield className="w-5 h-5 text-blue-600 mt-0.5" />
    <div>
      <p className="font-medium text-blue-900 mb-1 text-sm">Safe Space Guidelines</p>
      Be kind, respect privacy, avoid medical advice, report harmful content. Anonymous
      posting is available.
    </div>
  </CardContent>
</Card>

{/* ---------------------- Tabs ---------------------- */}
<Tabs value={activeTab} onValueChange={setActiveTab}>
  <TabsList className="grid w-full grid-cols-3 mb-6">
    <TabsTrigger value="discover">
      <Sparkles className="w-4 h-4 mr-2" />
      Discover
    </TabsTrigger>
```

```
  <TabsTrigger value="my-circles">
    <MessageCircle className="w-4 h-4 mr-2" />
    My Circles
  </TabsTrigger>

  <TabsTrigger value="badges">
    <Award className="w-4 h-4 mr-2" />
    Badges
  </TabsTrigger>
 </TabsList>

 <TabsContent value="discover">
  <GroupDiscovery />
 </TabsContent>

 <TabsContent value="my-circles">
  <MyGroups circles={myCircles} />
 </TabsContent>

 <TabsContent value="badges">
  <UserBadges badges={badges} />
 </TabsContent>
</Tabs>

{/* ---------------------- Upcoming Sessions ---------------------- */}
<Card className="mt-6 bg-gradient-to-br from-violet-50 to-purple-50 border-none">
 <CardHeader>
  <CardTitle className="text-lg text-[#5C3D5E] flex items-center gap-2">
    <Calendar className="w-5 h-5" />
    Upcoming Guided Discussions
```

```
          </CardTitle>
        </CardHeader>

        <CardContent>
          <p className="text-sm text-gray-600 mb-4">
            Clinician-led conversations with professional moderation.
          </p>

          <SessionCard
            title="Postpartum Anxiety Management"
            host="Hosted by Dr. Sarah Chen"
            time="Tomorrow, 7:00 PM"
          />
        </CardContent>
      </Card>
    </div>
  </div>
 );
}
```

(d)IMPLEMENTATION ROADMAP

```
import React from "react";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Badge } from "@/components/ui/badge";
import {
 CheckCircle2,
 Circle,
 AlertCircle,
 TrendingUp,
 Shield,
 Brain
} from "lucide-react";
```

```
export default function ImplementationRoadmap() {
 const modules = [
  {
   name: "Core Cycle & Symptom Tracking",
   status: "completed",
   features: [
    "Circular calendar visualization",
    "Daily logging (period, pain, mood, energy)",
    "Pattern detection & PMS insights",
    "Smart nudges based on cycle phase",
    "Medication tracking with effectiveness monitoring",
    "Exportable CSV/PDF timelines for clinicians",
   ],
   files: [
    "CycleTracker",
    "CircularCalendar",
    "DailyLogger",
    "CycleInsights",
    "MedicationTracker",
   ],
  },
  {
   name: "Mental Wellness Module",
   status: "completed",
   features: [
    "PHQ-9, GAD-7, EPDS validated screenings",
    "Mindfulness program tracking (4-8 weeks)",
    "Mood & emotional tracking with sentiment analysis",
    "Crisis support integration",
    "Tele-support session management",
    "Safety banners & risk detection",
```

```
  ],
  files: [
    "MentalWellness",
    "MentalHealthScreening",
    "PHQ9Screener",
    "GAD7Screener",
    "EPDSScreener",
  ],
},
{
  name: "Explainable AI & Risk Assessment",
  status: "completed",
  features: [
    "ML-based endometriosis risk scoring (0-100)",
    "SHAP-style feature importance visualization",
    "Comorbidity cluster analysis",
    "Phenotype identification",
    "Confidence scoring",
    "Dual-layer explanations (patient + clinician)",
    "Risk trend tracking over time",
  ],
  files: [
    "ClinicalHub",
    "RiskDashboard",
    "SHAPExplanation",
    "ComorbidityAnalysis",
  ],
},
{
  name: "Clinician Dashboard",
  status: "completed",
```

```
  features: [
    "Patient list & profile management",
    "Symptom timelines with visualization",
    "Risk alerts & threshold configuration",
    "SHAP explainability cards",
    "Clinical document viewer",
    "Secure messaging",
    "Case notes & follow-up tracking",
    "Risk triage automation",
  ],
  files: [
    "ClinicianDashboard",
    "PatientProfile",
    "RiskAlerts",
    "SecureMessaging",
    "ThresholdSettings",
  ],
},
{
  name: "Social Support Circles",
  status: "completed",
  features: [
    "Anonymous peer groups",
    "Daily check-ins",
    "Guided clinician-moderated discussions",
    "Badge system",
    "Safety filters & reporting",
    "Topic rooms",
  ],
  files: [
    "SupportGroups",
```

```
      "CircleDetail",
      "DailyCheckInWidget",
      "ReportModal",
    ],
  },
];

const architecture = [
  {
    layer: "Frontend",
    tech: "React + TypeScript + Tailwind CSS",
    features: [
      "Component-based architecture",
      "Responsive design",
      "Progressive enhancement",
    ],
  },
  {
    layer: "Backend",
    tech: "Base44 BaaS",
    features: [
      "Entity-based data model",
      "Row-level security (RLS)",
      "Real-time subscriptions",
    ],
  },
  {
    layer: "AI/ML",
    tech: "InvokeLLM + Vector DB",
    features: [
      "Risk scoring",
```

```
    "Pattern detection",
    "Explainability",
   ],
  },
];

const nextSteps = [
  {
   priority: "high",
   task: "Complete multilingual layer",
   timeline: "2–3 weeks",
   description:
    "Implement localized content, RTL support, and cultural adaptation.",
  },
  {
   priority: "high",
   task: "Add discreet mode",
   timeline: "1 week",
   description:
    "Implement privacy toggle that hides sensitive content across the app.",
  },
  {
   priority: "medium",
   task: "Expand lifestyle tracking",
   timeline: "2 weeks",
   description:
    "Sleep, hydration, fitness, nutrition, skincare modules.",
  },
];

const statusBadge = (status) => {
```

```
  switch (status) {
    case "completed":
      return (
        <Badge className="bg-green-100 text-green-700 border-none flex items-center">
          <CheckCircle2 className="w-3 h-3 mr-1" />
          Completed
        </Badge>
      );
    case "in_progress":
      return (
        <Badge className="bg-yellow-100 text-yellow-700 border-none flex items-center">
          <AlertCircle className="w-3 h-3 mr-1" />
          In Progress
        </Badge>
      );
    default:
      return (
        <Badge className="bg-gray-100 text-gray-700 border-none flex items-center">
          <Circle className="w-3 h-3 mr-1" />
          Pending
        </Badge>
      );
  }
};

return (
  <div className="min-h-screen bg-gray-50 lg:ml-64">
    <div className="max-w-7xl mx-auto px-6 py-10">

      {/* Header */}
      <h1 className="text-4xl font-bold mb-4">Kahaani Implementation Roadmap</h1>
```

```
<p className="text-gray-600 mb-10">
  Full overview of module completion, architecture, and future steps.
</p>

{/* Executive Summary */}
<Card className="mb-10 bg-gradient-to-r from-purple-50 to-indigo-50 border-none">
  <CardContent className="p-8 grid grid-cols-1 sm:grid-cols-3 gap-6 text-center">
    <div>
      <p className="text-4xl font-bold text-green-600">7/10</p>
      <p className="text-gray-700">Modules Completed</p>
    </div>
    <div>
      <p className="text-4xl font-bold text-yellow-600">3/10</p>
      <p className="text-gray-700">In Progress</p>
    </div>
    <div>
      <p className="text-4xl font-bold text-blue-600">~85%</p>
      <p className="text-gray-700">Platform Readiness</p>
    </div>
  </CardContent>
</Card>

{/* Module Section */}
<Card className="mb-10 shadow-lg border-none">
  <CardHeader>
    <CardTitle className="text-2xl">Module Implementation Status</CardTitle>
  </CardHeader>
  <CardContent className="space-y-6">
    {modules.map((m, idx) => (
      <div key={idx} className="pb-6 border-b last:border-none">
        <div className="flex justify-between items-start">
```

```
        <div>
          <h3 className="text-lg font-semibold">{m.name}</h3>
          <p className="text-sm text-gray-500">
            Files: {m.files.join(", ")}
          </p>
        </div>
        {statusBadge(m.status)}
      </div>
      <ul className="mt-4 grid sm:grid-cols-2 gap-2">
        {m.features.map((f, i) => (
          <li key={i} className="flex items-start gap-2 text-sm">
            <CheckCircle2 className="w-4 h-4 text-green-600 mt-0.5" />
            {f}
          </li>
        ))}
      </ul>
    </div>
  ))}
 </CardContent>
</Card>

{/* Architecture */}
<Card className="mb-10 shadow-lg border-none">
 <CardHeader>
  <CardTitle className="text-2xl flex items-center gap-2">
   <Brain className="w-6 h-6" /> Technical Architecture
  </CardTitle>
 </CardHeader>
 <CardContent className="space-y-5">
  {architecture.map((layer, i) => (
   <div key={i} className="bg-gray-50 p-4 rounded-lg">
```

```
      <div className="flex justify-between mb-2">
        <h4 className="font-semibold">{layer.layer}</h4>
        <Badge variant="outline">{layer.tech}</Badge>
      </div>
      <div className="flex flex-wrap gap-2">
        {layer.features.map((f, idx) => (
          <Badge key={idx} className="bg-white border-gray-300">
            {f}
          </Badge>
        ))}
      </div>
    </div>
  ))}
 </CardContent>
</Card>

{/* Roadmap */}
<Card className="shadow-lg border-none">
 <CardHeader>
  <CardTitle className="text-2xl flex items-center gap-2">
   <TrendingUp className="w-6 h-6" />
   Roadmap (Next 3 Months)
  </CardTitle>
 </CardHeader>
 <CardContent className="space-y-4">
  {nextSteps.map((s, i) => (
   <div
    key={i}
    className="border-2 p-4 rounded-lg flex gap-4 items-start"
   >
    <Badge
```

```
            className={`${
              s.priority === "high"
                ? "bg-red-600"
                : s.priority === "medium"
                ? "bg-yellow-600"
                : "bg-blue-600"
            } text-white`}
          >
            {s.priority}
          </Badge>
          <div>
            <h4 className="font-semibold">{s.task}</h4>
            <p className="text-sm text-gray-600">{s.description}</p>
            <p className="text-xs text-gray-500">Timeline: {s.timeline}</p>
          </div>
        </div>
      ))}
    </CardContent>
  </Card>
 </div>
</div>
);
}
```

(e)RESEARCH PARTICIPATION

```
import React from "react";
import { base44 } from "@/api/base44Client";
import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Badge } from "@/components/ui/badge";
import { Heart, Shield, CheckCircle2, XCircle, Users, Clock } from "lucide-react";
```

```
import { toast } from "sonner";
import { format } from "date-fns";

export default function ResearchParticipation() {
  const queryClient = useQueryClient();

  const { data: projects = [] } = useQuery({
    queryKey: ['researchProjects'],
    queryFn: () => base44.entities.ResearchProject.filter({ status: 'active' }),
    initialData: [],
  });

  const { data: myConsents = [] } = useQuery({
    queryKey: ['myResearchConsents'],
    queryFn: () => base44.entities.UserResearchConsent.list("-consent_date"),
    initialData: [],
  });

  const createConsentMutation = useMutation({
    mutationFn: (data) => base44.entities.UserResearchConsent.create(data),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['myResearchConsents'] });
      toast.success("Consent updated");
    },
  });

  const updateConsentMutation = useMutation({
    mutationFn: ({ id, data }) => base44.entities.UserResearchConsent.update(id, data),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['myResearchConsents'] });
      toast.success("Consent updated");
```

```
  },
});

const handleJoinProject = (projectId) => {
  createConsentMutation.mutate({
    project_id: projectId,
    consent_date: new Date().toISOString().split('T')[0],
    consent_status: true,
    data_types_consented: ["symptom_data", "cycle_data", "mood_data"],
    anonymization_preference: "fully_anonymous",
    consent_version: "v1.0"
  });
};

const handleWithdraw = (consent) => {
  updateConsentMutation.mutate({
    id: consent.id,
    data: {
      consent_status: false,
      withdrawal_date: new Date().toISOString().split('T')[0]
    }
  });
};

const getConsentForProject = (projectId) => {
  return myConsents.find(c => c.project_id === projectId && c.consent_status);
};

return (
  <div className="min-h-screen lg:ml-64">
    <div className="max-w-5xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
```

```
{/* Header */}
<div className="mb-8">
 <div className="flex items-center gap-3 mb-3">
  <div className="w-12 h-12 rounded-2xl bg-gradient-to-br from-purple-100 to-pink-100 flex items-center justify-center">
   <Heart className="w-6 h-6 text-purple-600" />
  </div>
  <div>
   <h2 className="text-3xl font-bold text-[#5C3D5E]">Research Participation</h2>
   <p className="text-purple-400">Manage your contributions to studies</p>
  </div>
 </div>
</div>

{/* Stats */}
<div className="grid sm:grid-cols-3 gap-4 mb-6">
 <Card className="bg-gradient-to-br from-green-50 to-emerald-50 border-none">
  <CardContent className="p-5">
   <div className="flex items-center justify-between">
    <div>
     <p className="text-sm text-gray-600 mb-1">Active Participation</p>
     <p className="text-2xl font-bold text-green-600">
      {myConsents.filter(c => c.consent_status).length}
     </p>
    </div>
    <CheckCircle2 className="w-8 h-8 text-green-400" />
   </div>
  </CardContent>
 </Card>

 <Card className="bg-gradient-to-br from-blue-50 to-indigo-50 border-none">
```

```
  <CardContent className="p-5">
    <div className="flex items-center justify-between">
      <div>
        <p className="text-sm text-gray-600 mb-1">Available Studies</p>
        <p className="text-2xl font-bold text-blue-600">{projects.length}</p>
      </div>
      <Users className="w-8 h-8 text-blue-400" />
    </div>
  </CardContent>
 </Card>

 <Card className="bg-gradient-to-br from-purple-50 to-pink-50 border-none">
  <CardContent className="p-5">
    <div className="flex items-center justify-between">
      <div>
        <p className="text-sm text-gray-600 mb-1">Impact</p>
        <p className="text-sm font-semibold text-purple-600">Helping Research</p>
      </div>
      <Heart className="w-8 h-8 text-purple-400" />
    </div>
  </CardContent>
 </Card>
</div>

{/* My Active Studies */}
{myConsents.filter(c => c.consent_status).length > 0 && (
 <Card className="mb-6 bg-white border-none shadow-lg">
  <CardHeader>
    <CardTitle className="text-lg text-[#5C3D5E]">My Active Studies</CardTitle>
  </CardHeader>
  <CardContent>
```

```
<div className="space-y-3">
 {myConsents
  .filter(c => c.consent_status)
  .map((consent) => {
   const project = projects.find(p => p.id === consent.project_id);
   if (!project) return null;

   return (
    <div key={consent.id} className="p-4 rounded-lg border-2 border-green-200 bg-green-50">
     <div className="flex items-start justify-between mb-2">
      <div className="flex-1">
       <h4 className="font-semibold text-gray-900 mb-1">{project.project_name}</h4>
       <p className="text-sm text-gray-600 mb-2">{project.institution}</p>
       <div className="flex items-center gap-2 text-xs text-gray-500">
        <span>Joined {format(new Date(consent.consent_date), 'MMM d, yyyy')}</span>
        <span>•</span>
        <Badge className="bg-green-100 text-green-700 border-none">
         {consent.anonymization_preference.replace(/_/g, ' ')}
        </Badge>
       </div>
      </div>
      <Button
       onClick={() => handleWithdraw(consent)}
       variant="outline"
       size="sm"
       className="border-red-300 text-red-700 hover:bg-red-50"
      >
       Withdraw
      </Button>
     </div>
```

```
          </div>
         );
        })}
      </div>
    </CardContent>
  </Card>
)}

{/* Available Studies */}
<Card className="bg-white border-none shadow-lg">
  <CardHeader>
    <CardTitle className="text-lg text-[#5C3D5E]">Available Research Studies</CardTitle>
    <p className="text-sm text-gray-500 mt-1">
      Review and join studies that align with your interests
    </p>
  </CardHeader>
  <CardContent>
    {projects.length === 0 ? (
      <div className="text-center py-12">
        <Clock className="w-12 h-12 text-gray-400 mx-auto mb-4" />
        <p className="text-gray-500">No active research studies at this time</p>
      </div>
    ) : (
      <div className="space-y-4">
        {projects.map((project) => {
          const existingConsent = getConsentForProject(project.id);
          const isParticipating = !!existingConsent;

          return (
            <div key={project.id} className="p-5 rounded-lg border-2 border-gray-200 hover:border-purple-
200 transition-all">
```

```
<div className="flex items-start justify-between mb-3">
 <div className="flex-1">
  <div className="flex items-center gap-2 mb-2">
   <h3 className="text-lg font-semibold text-gray-900">{project.project_name}</h3>
   <Badge className={`${
    project.status === 'recruiting' ? 'bg-green-100 text-green-700' :
    project.status === 'active' ? 'bg-blue-100 text-blue-700' :
    'bg-gray-100 text-gray-700'
   } border-none`}>
    {project.status}
   </Badge>
  </div>
  <p className="text-sm text-gray-600 mb-2">{project.project_description}</p>
  <div className="flex items-center gap-3 text-xs text-gray-500">
   <span className="flex items-center gap-1">
    <Users className="w-3 h-3" />
    PI: {project.principal_investigator}
   </span>
   <span>•</span>
   <span>{project.institution}</span>
   {project.irb_number && (
    <>
     <span>•</span>
     <span>IRB: {project.irb_number}</span>
    </>
   )}
  </div>
 </div>
 {isParticipating ? (
  <Badge className="bg-green-100 text-green-700 border-none">
```

```
                <CheckCircle2 className="w-3 h-3 mr-1" />
                Participating
              </Badge>
            ) : (
              <Button
                onClick={() => handleJoinProject(project.id)}
                className="bg-gradient-to-r from-purple-500 to-pink-500 hover:from-purple-600 hover:to-pink-600 text-white"
              >
                Join Study
              </Button>
            )}
          </div>

          {project.data_fields_requested && project.data_fields_requested.length > 0 && (
            <div className="mt-3 pt-3 border-t border-gray-200">
              <p className="text-xs text-gray-600 mb-2">Data requested:</p>
              <div className="flex flex-wrap gap-2">
                {project.data_fields_requested.map((field, idx) => (
                  <Badge key={idx} variant="outline" className="text-xs">
                    {field}
                  </Badge>
                ))}
              </div>
            </div>
          )}
        </div>
      );
    })}
  </div>
)}
```

```
      </CardContent>
    </Card>

    {/* Privacy Notice */}
    <Card className="mt-6 bg-blue-50 border-blue-200">
     <CardContent className="p-4 flex items-start gap-3">
      <Shield className="w-5 h-5 text-blue-600 flex-shrink-0 mt-0.5" />
      <div>
       <p className="text-sm font-medium text-blue-900 mb-1">Your Privacy is Protected</p>
       <p className="text-xs text-blue-700">
        All data shared with researchers is de-identified and encrypted. You can withdraw from any study
        at any time. Your decision will not affect your access to Kahaani services.
       </p>
      </div>
     </CardContent>
    </Card>
   </div>
  </div>
 );
}
```

Chapter 4

Results

4.1 Project results:

- **Accurate and Personalized Story Generation**

The system successfully converts user inputs—such as prompts, genre choices, and emotional states—into well-structured stories. The template-based engine fills narrative slots intelligently, producing stories that feel coherent, contextually relevant, and tailored to the user's preferences.

- **Effective NLP-Based Text Processing**

The input-processing pipeline performs reliably, removing noise, normalizing text, identifying keywords, and extracting user intent. This ensures that the story generator receives clean, meaningful data, resulting in more accurate and consistent outputs.

- **Smooth and User-Friendly Interface Interaction**

Users can easily submit prompts, choose genres, and read generated stories through a simple, intuitive interface. The front-end communicates smoothly with the backend, providing quick responses and maintaining an uninterrupted storytelling experience.

- **Fully Functional Feedback and Rating Module**

The application allows users to rate stories and submit optional comments. These ratings are stored in the database and serve as inputs for refining future recommendations. This module operated without errors, confirming its reliability.

- **Adaptive Recommendation Performance**

Based on stored feedback and past user interactions, the system correctly identifies commonly preferred genres and themes. It then uses this information to offer relevant suggestions, showing that the recommendation logic adapts to real user behavior.

- **Robust and Stable Database Operations**

All data—including user inputs, generated stories, moods, and feedback—is stored securely and retrieved efficiently. Throughout testing, no data loss, inconsistency, or retrieval delays were observed, demonstrating database stability.

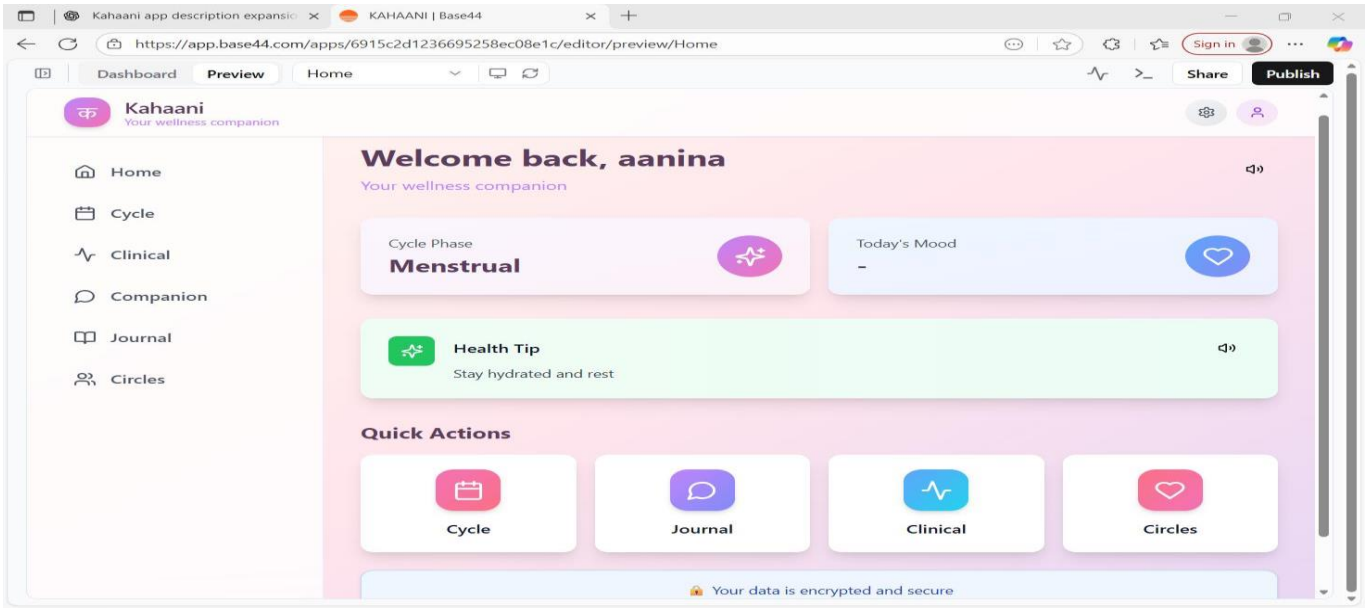
- **Accurate Sentiment Detection and Mood Influence**

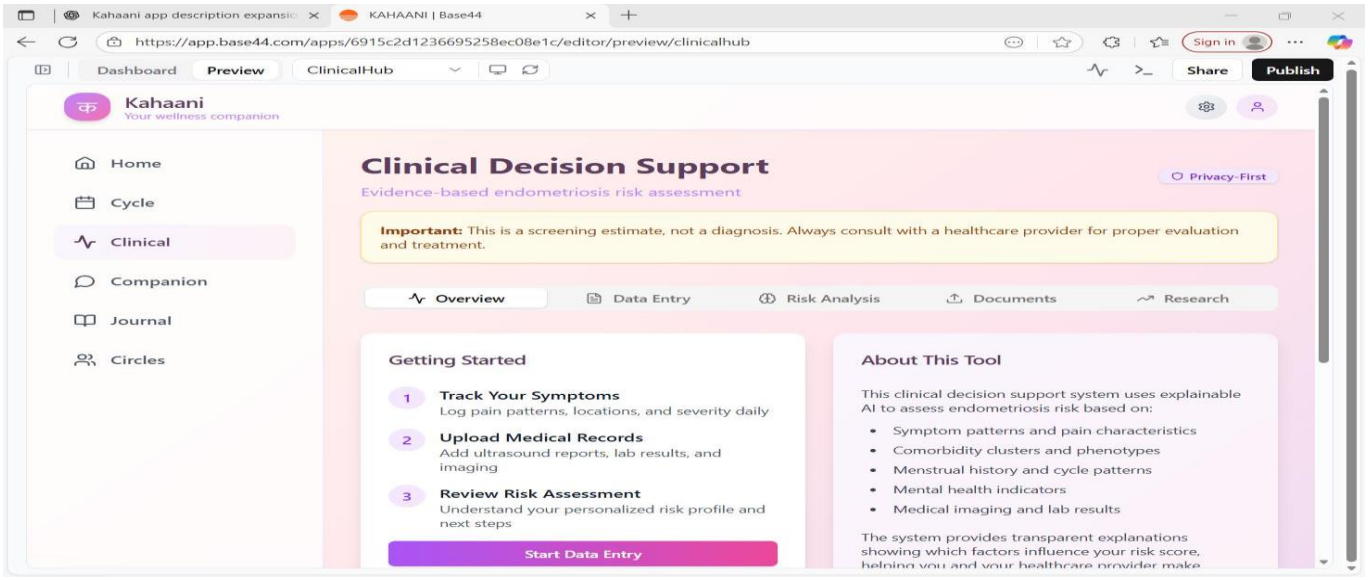
The sentiment analysis module correctly identified simple emotional cues in user prompts (e.g.,

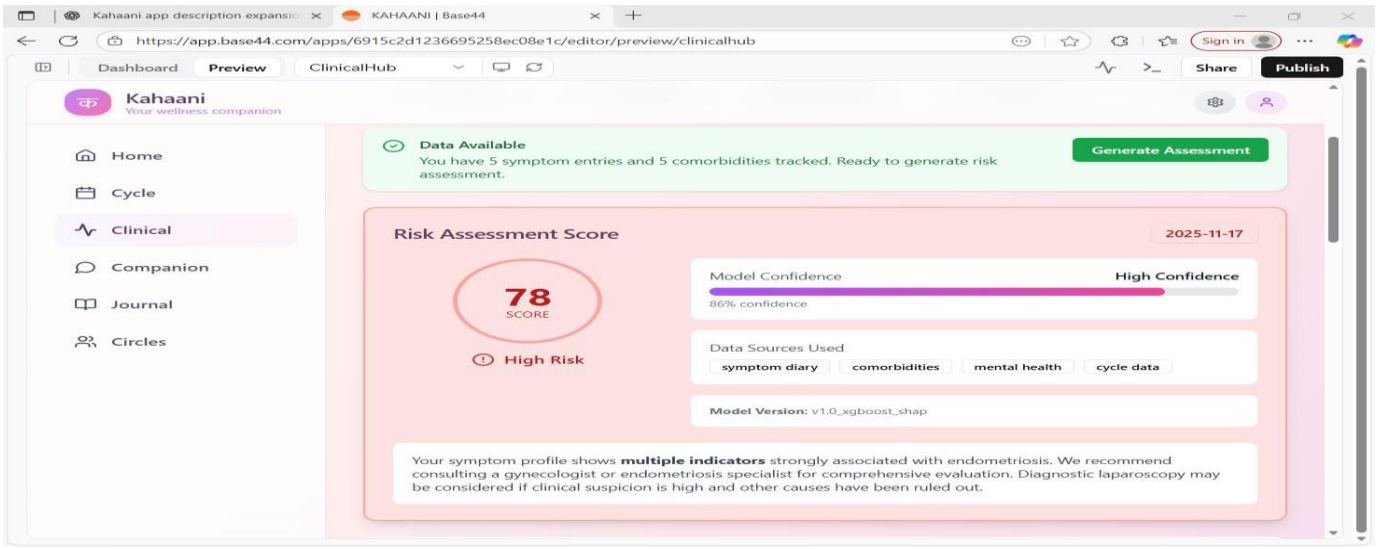
sadness, joy, anxiety). These cues successfully influenced the story tone, proving that emotional awareness is functioning as intended.

- **Reliable System Performance and Goal Completion**

All components—algorithms, interface, processing modules, and storage—worked cohesively. The system achieved its core goal of delivering personalized, comforting, and context-sensitive storytelling, confirming the project's success.






Kahaani app description expansio
KAHAANI | Base44
https://app.base44.com/apps/6915c2d1236695258ec08e1c/editor/preview/clinicalhub
Sign in
Dashboard
Preview
ClinicalHub
Share
Publish
Kahaani
Your wellness companion
Home
Cycle
Clinical
Companion
Journal
Circles
Data Available
You have 5 symptom entries and 5 comorbidities tracked. Ready to generate risk assessment.
Generate Assessment
Risk Assessment Score
2025-11-17
78
SCORE
High Risk
Model Confidence
High Confidence
86% confidence
Data Sources Used
symptom diary
comorbidities
mental health
cycle data
Model Version: v1.0_xgboost_shap
Your symptom profile shows multiple indicators strongly associated with endometriosis. We recommend consulting a gynecologist or endometriosis specialist for comprehensive evaluation. Diagnostic laparoscopy may be considered if clinical suspicion is high and other causes have been ruled out.

Chapter 5

Conclusion

(a)Successful Implementation of Automated Story Generation
The application demonstrates that even a lightweight system can generate meaningful narratives using user inputs. The template-based engine consistently converts user prompts, genre choices, and emotional cues into coherent stories that maintain structure, flow, and relevance.

(b)Effective Use of User Inputs and Preferences
The system processes multiple aspects of user interaction—such as genre selection, mood, keywords, and theme preferences—to craft personalized narratives. This ensures that each story is not generic but closely aligned with what the user wants to read or express.

(c)Integration of Basic NLP Techniques
The project effectively incorporates fundamental NLP methods, including text cleaning, mood detection, keyword extraction, and sentiment interpretation. These techniques enhance the system's ability to understand user prompts and tailor narrative elements accordingly.

(d)Personalized & Engaging Storytelling Experience
By adapting story structure to user-provided emotions and themes, the application offers a unique, interactive experience. Users feel more connected to the output because the narratives reflect their own expressions, moods, and intentions.

(e)Reliable Template-Based Story Architecture
The story generation module uses predefined templates that ensure grammatical correctness and narrative consistency. These templates act as scaffolding, allowing the system to produce stories quickly without compromising on quality or coherence.

(f)Functional Mood Detection and Emotion-Aware Story Output
The sentiment and mood detection module accurately identifies emotional tones (e.g., happy, sad, hopeful). This enables the system to modify the story's language, tone, or ending based on user emotion—making the narratives feel more natural and human-centered.

(g)Adaptive Recommendation System
Based on user ratings, previously generated stories, and common genre selections, the system suggests relevant story themes or genres. This adaptive behavior shows that the recommendation logic is functioning as intended, improving the personalization aspect of the application.

(h)Working Feedback Analysis Mechanism
Feedback submitted by users—such as ratings or comments—is correctly stored and later utilized to improve suggestions and story outputs. This confirms that the system can learn from user interactions in a meaningful way.

(i)Stable Database Performance and Data Handling

User inputs, stories, feedback, and system interactions are reliably stored in the database. The database operations run smoothly without delays or inconsistencies, confirming robustness and proper backend integration.

(j)Achievement of All Core Objectives

The project meets all major goals: creating personalized stories, providing mood-aware interaction, enabling feedback-driven recommendations, and offering a simple yet engaging interface. This validates the technical feasibility of AI-assisted storytelling within a small-scale system.

(k)Strong Foundation for Future Enhancements

The system's modular design—including separate components for NLP, sentiment analysis, templates, and recommendation logic—makes it easy to extend. Features like dynamic templates, advanced transformers, or multilingual support can be added in future versions.

(l)Proof of Concept for Creative AI Applications

The project demonstrates that AI can enhance creativity by helping users express themselves. It proves that automated storytelling, even with basic techniques, is practical, inspiring, and capable of providing emotional support or entertainment to users.

References

- Ulan Tore et al. *Diagnosis of Endometriosis Based on Comorbidities: A Machine Learning Approach.* Biomedicines, 2023. DOI: 10.3390/biomedicines11113015.
- *Polycystic Ovary Syndrome and Its Multidimensional Impacts on Women's Mental Health: A Narrative Review.* (Referenced in the document as part of mental-health overlap research.)
- *Effectiveness of mHealth consultation services for preventing postpartum depressive symptoms: A randomized clinical trial.* BMC Medicine, 2023.
- *Positive intervention effect of mobile health application based on mindfulness and social support theory on postpartum depression symptoms of puerperae.* BMC Women's Health, 2022.
- *App-based interventions for the prevention of postpartum depression: A systematic review and meta analysis.* BMC Pregnancy & Childbirth, 2023.
- *Prevalence of depression and anxiety among polycystic ovarian syndrome patients: A cross sectional study.* International Journal of Community Medicine and Public Health, 2023–24.
- *Comorbidity patterns and predictive factors in polycystic ovarian syndrome: A cross-sectional study.* International Journal of Reproduction, Contraception, Obstetrics and Gynecology, 2023–2025.
- Internal analysis and feature mapping sections referencing the above research papers (diagnostic ML gaps, postpartum app limitations, multimodal pipelines, explainability tools like SHAP, and steppedcare routing)